ELEC3104: Digital Signal Processing Lab

Dr Phu Le & Professor Eliathamby Ambikairajah
School of Electrical Engineering and Telecommunications
UNSW, Sydney, Australia

Term 3, 2021

Topic 2 – Convolution

Exercise: Given $x_1[n] = \delta[n] + 2\,\delta[n-1] + 3\delta[n-2] + 4\delta[n-3]$

$$x_2[n] = \delta[n-1] - 2\,\delta[n-2]$$

a. Find $x_1[n]*x_2[n]$
b. Use MATLAB command "conv" to check the results.

```
x1=[1 2 3 4];
x2 = [0 1 -2];
y = conv(x1,x2);
```

Exercise: In MATLAB, write a function *my_conv* that takes two inputs x1, x2 and produces the output y equal to the convolution of x1 and x2. Compare the result produced by your *my_conv* function and the MATLAB in-built function *conv*

$$y[n] = x_1[n] * x_2[n] = \sum_{k=-\infty}^{+\infty} x_1[n-k].x_2[k]$$

```
function y = my_conv(x1, x2)
x1f=fliplr(x1);%flip left right
len_x1 = length(x1);
x2e=[zeros(1,len_x1-1) x2 zeros(1,len_x1-1)];%padding zero
for i=1:len_x1+length(x2)-1
    y(i)=sum(x1f.*x2e(i:i+len_x1-1));
end
```

Topic 2 – Discrete Time System Implementation

Exercise: Given a system described by the following block diagram. In MATLAB, compute its output y when the input signal is `x = [1 -3 2 4 -2 2 6 2 4 -2]` using three different methods as given below:

a. Using a for loop

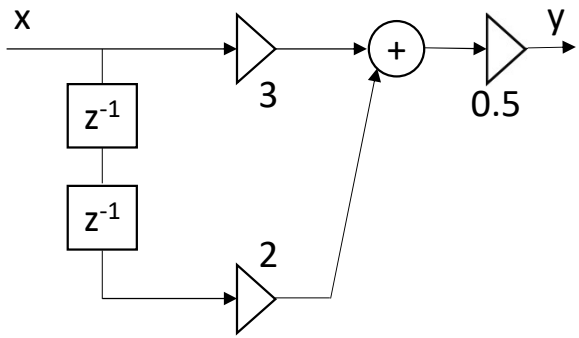
```
x = [1 -3 2 4 -2 2 6 2 4 -2];
xe = [x 0 0]; %extended x
yL(1) = 3*xe(1)*0.5;
yL(2) = 3*xe(2)*0.5;
for i=3:length(xe)
    yL(i) = (3*xe(i)+2*xe(i-2))*0.5;
end
```

b. 'conv' command

```
yC = conv(x, [3 0 2])*0.5;
```

c. 'filter' command

```
[yF, Zf] = filter([3 0 2]*0.5, 1, x);
yF = [yF Zf'];
```



Compare the results of (a), (b) and (c)

Should be the same

Topic 2 – Pole-Zero Description of Discrete-Time Systems

Exercise: A discrete-time system described by its transfer function is shown below: $H(z) = \frac{z^8 - (0.95)^8}{z^7(z - 0.95)}$

Using MATLAB,

a. Find all the poles and zeros of the system and plot them in the z-plane

```
B = [1 zeros(1,7) -0.95^8];
A=[1 -0.95];
zeros = roots(B);
poles = roots(A);
zplane(zeros, poles)
```

b. Compute and plot the magnitude response of the system

```
[H,w]=freqz(B,A);
plot(w/w(end), abs(H));
xlabel('Normalized frequency')
ylabel('Magnitude')
```

c. Compute and plot the impulse response of the system

```
[h] = impz(B,A);
stem(0:length(h)-1,h)
xlabel('n')
ylabel('h[n]')
axis([0 20 0 max(h)+0.1])
```

Hint: The in-built functions *freqz* and *impz* could be used to compute the frequency and impulse responses

Topic 2 – Pole-Zero Description of Discrete-Time Systems (2)

Exercise: A 2nd order system has two zeros at the origin and two poles as $p_1=re^{j\pi/4}$, $p_2 = re^{-j\pi/4}$, described by the following transfer function

$$H(z) = \frac{z^2}{(z-p_1)(z-p_2)}$$

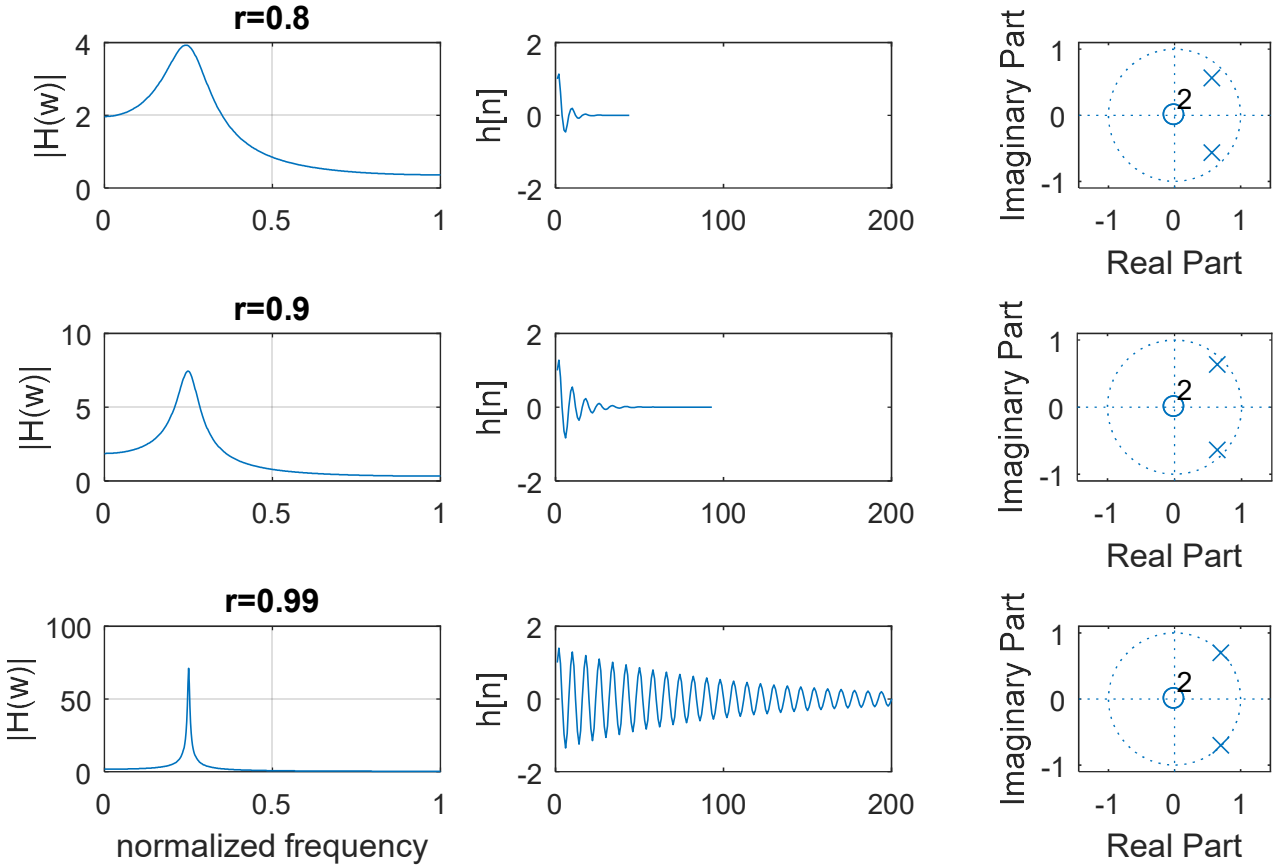
a. Using MATLAB, compute and plot the magnitude response, impulse response, and pole-zero plane (z-plane) of three different 2nd order systems.
   System 1: r = 0.8
   System 2: r = 0.9
   System 3: r = 0.99

b. Comment on the impact of the location of poles to the impulse response and magnitude response for three systems of (a) ->*system with poles closer to unit circle will have shaper peak at resonant frequency and impulse response decays slower*

```
theta = pi/4;
B = 1;
rs = [0.8, 0.9, 0.99];
for i=1:length(rs)
    r=rs(i);
    A = [1 -2*r*cos(theta) r^2];
    [H, w] = freqz(B,A);
    subplot(3,3,1+(i-1)*3)
    plot(w/w(end), abs(H))
    ylabel('|H(w)|')
    title(['r=' num2str(r)])
    grid on
    subplot(3,3,2+(i-1)*3)
    plot(impz(B,A))
    ylabel('h[n]')
    axis([0 200 -2 2])
    subplot(3,3,3+(i-1)*3)
    zplane(B,A)
end
subplot 337
xlabel('normalized frequency')
```


5

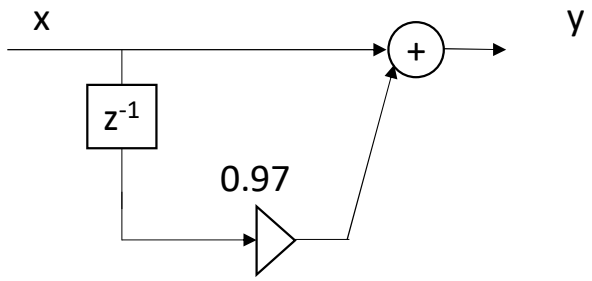
Topic 2 – Pole-Zero Description of Discrete-Time Systems (2)



Topic 2 z-Transform, Impulse Response and Frequency Response

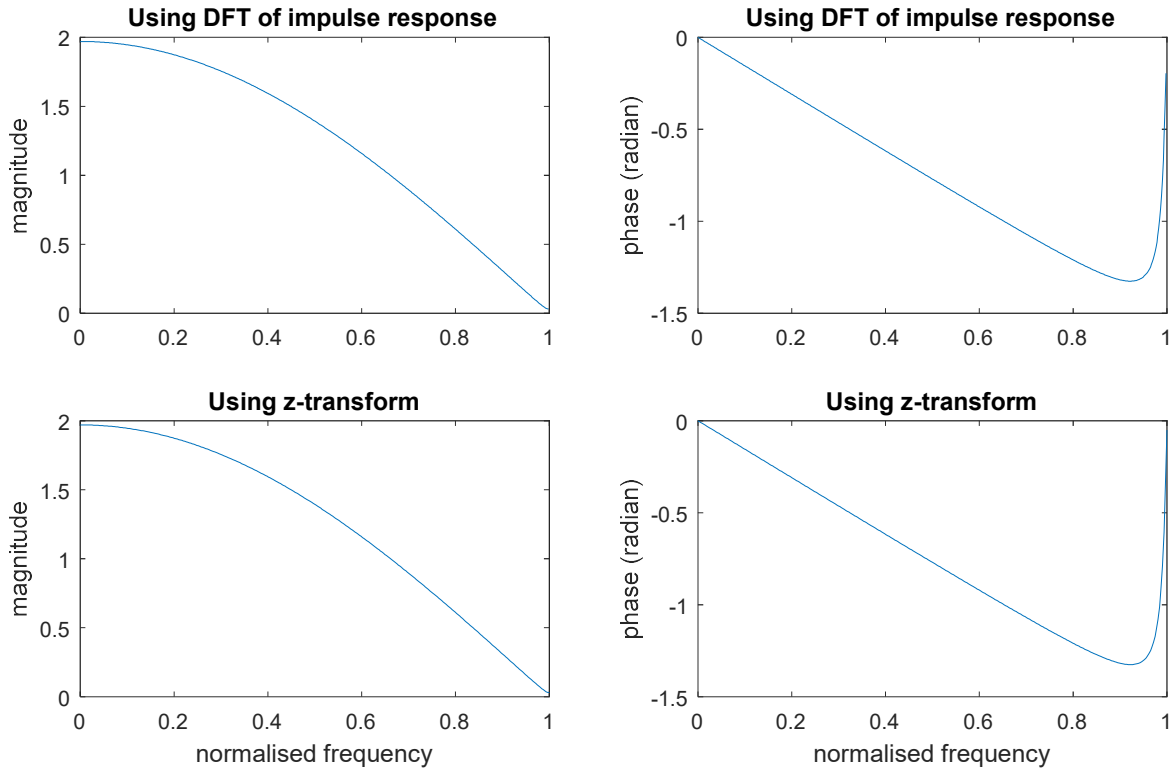
Exercise: There are two different methods to compute the frequency response of a discrete-time system. The first method is using the z-transform and the second one is using Discrete Fourier Transform (DFT) of its impulse response

In MATLAB, write a program to compute the frequency response of the following system and compare the magnitude and phase responses obtained by the above two methods .




```
h = [1 0.97]; NFFT=1024;
freq_axis = (0:NFFT/2-1)/(NFFT/2);%normalised frequency axis in [0 fs/2]
imp = [1 zeros(1,NFFT-1)];% impulse or delta signal
imp_res = conv(h, imp); %system impulse response
freq_res_DFT = fft(imp_res, NFFT); %DFT of impulse response from 0 to fs
freq_res_DFT = freq_res_DFT(1:NFFT/2);%DFT of impulse resp. in [0 fs/2]
%%%%%from z-transform%H(z)=1+0.97z^-1
theta = 0:0.01:pi;
H=1+0.97*exp(-j*theta);
figure
subplot 221
plot(freq_axis, abs(freq_res_DFT))
ylabel('magnitude')
title('Using DFT of impulse response')
subplot 223
plot(theta/theta(end),abs(H))
ylabel('magnitude'); xlabel('normalised frequency')
title('Using z-transform')
subplot 222
plot(freq_axis, angle(freq_res_DFT))
ylabel('phase (radian)')
title('Using DFT of impulse response')
subplot 224
plot(theta/theta(end), angle(H))
xlabel('normalised frequency'); ylabel('phase (radian)')
title('Using z-transform')
```

Topic 2 z-Transform, Impulse Response and Frequency Response



Topic 2 – Discrete Fourier Transform (DFT)

Exercise: Given a discrete time sinusoid signal saved in a wav file '*signal1.wav*' located in Moodle folder Additional Resources/ Wav files for DSP Labs. Using MATLAB,

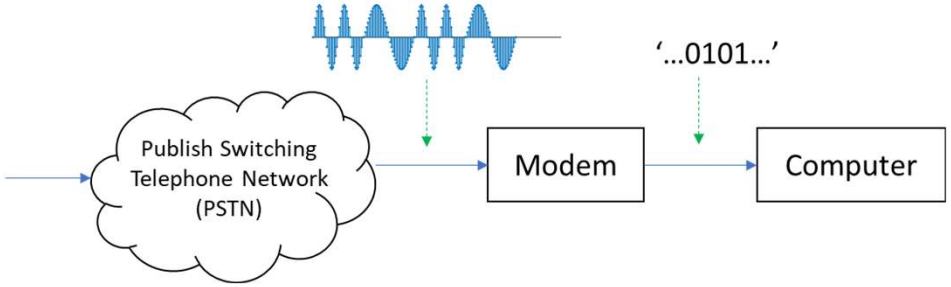
a. Write a function *frequency_estimation* to read this wav file, automatically estimate and return the frequency of the sinusoid signal.
   -> estimated frequency should be ≈ 950Hz
b. Evaluate the estimation accuracy of your algorithm in (a)
   -> estimation error can go up to fs/NFFT (Hz)

Hint: The in-built functions *audioread* could be used to read the wav file and signal sampling rate.

```
function est_freq=frequency_estimation(wavFile)
[x,fs]=audioread('signal1.wav');
NFFT=1024;
freqHz=(0:NFFT-1)/NFFT*fs;
freqHz=freqHz(1:NFFT/2);
xs=x(1:NFFT);
X=fft(xs,NFFT);
X=abs(X(1:NFFT/2));
% % % plot(freqHz, X);
i=2;
while i<length(X)-1
    if (X(i)>X(i-1))&(X(i)>X(i+1))
        peak_id=i;
        break
    end
    i=i+1;
end
est_freq=freqHz(peak_id);
```

Topic 2 – Discrete Fourier Transform (DFT)

Exercise: In ancient times, modems worked by transmitting digital data through telephone lines. Specifically, during transmission modems converted a stream of 1's and 0's into an audio signal where 1's were represented by a particular frequency and 0's by another frequency. A modem acting as a receiver would convert the audio signal back into a stream of 1's and 0's. You task today is to design and implement a system in MATLAB (as a MATLAB function) that will accept such audio signals (supplied to you as wav files – 'bit_sequence1.wav', 'bit_sequence2.wav', 'bit_sequence3.wav' and 'bit_sequence4.wav' located in Moodle folder Additional Resources/ Wav files for DSP Labs) as input and convert them into sequences of 1's and 0's accurately.



File	Decoded Bit Sequence
bit_sequence1.wav	
bit_sequence2.wav	
bit_sequence3.wav	
bit_sequence4.wav	

You have been given the following information about the audio signal:

1. Each symbol (i.e., 0 or 1) is represented by a sinusoid of a single frequency.
2. Each transmission begins with 0101.
3. The transmission rate is 20 bits/second

Topic 2 – Discrete Fourier Transform (DFT)

```
[AudioSeq,fs]=audioread('bit_sequence1.wav');
NFFT=1024;
FreqScalePoint=0:NFFT-1;
FreqScaleHz=fs*FreqScalePoint/NFFT; FreqScaleHz=FreqScaleHz(1:NFFT/2);
OneBitLen=fs/20; NoOfBits=length(AudioSeq)/OneBitLen;
BitSeq='';
for BitIndex=1:NoOfBits
    OneBitSegment=AudioSeq((BitIndex-1)*OneBitLen+1:BitIndex*OneBitLen);
    OneBitSegmentFFT=abs(fft(OneBitSegment,NFFT));
    OneBitSegmentFFT=OneBitSegmentFFT(1:NFFT/2);
    if BitIndex==1
        Bit0Freq=FreqScaleHz(find(OneBitSegmentFFT==max(OneBitSegmentFFT)));
        BitSeq=[BitSeq '0'];
    elseif BitIndex==2
        Bit1Freq=FreqScaleHz(find(OneBitSegmentFFT==max(OneBitSegmentFFT)));
         BitSeq=[BitSeq '1'];
    elseif BitIndex>2
        BitFreq=FreqScaleHz(find(OneBitSegmentFFT==max(OneBitSegmentFFT)));
        switch BitFreq
            case Bit1Freq
                BitSeq=[BitSeq '1'];
            case Bit0Freq
                 BitSeq=[BitSeq '0'];
        end
    end
end
BitSeq
```

File	Decode Bit Sequence
'bit_sequence1.wav	0101001111000110
'bit_sequence2.wav	010101000110
'bit_sequence3.wav	0101001100
'bit_sequence4.wav	010110111000110